BigDecimal v1.6 Add-in

Functional Understanding Document

Version 1.4

Table of Contents

1 Introduction 4

1.1 Why BigDecimal? 4

2 Key Features and Capabilities 4

3 The BigCalculator: Desktop-Grade Precision in Excel 5

4 Installation of BigDecimal v1.6.xlam 6

4.1.1 Downloading the BigDecimal Add-in 6

4.1.2 Important: Unblocking the Add-in 7

5 Working with BigDecimal 8

5.1 Referencing BigDecimal in VBA 8

6 BigDecimal "Quick Start" Guide 8

6.1 Instantiating the BigDecimal Class 9

6.2 Casting standard types 9

6.3 Implementation Example: Basic Addition 9

6.4 Language Limitations: Operator Overloading 10

7 Functional Specifications 10

7.1 BigDecimal Methods 11

7.2 BigDecimal Properties 12

7.2.1 The Default .Value Property 13

7.2.2 The .ValueEx Property 13

7.2.3 The .TarScale Property 14

7.2.4 Auto-Casting of Standard Types 15

7.2.5 Effortless Precision with Auto-Trimming 16

7.2.6 Mutable vs. Immutable Properties 16

7.3 Global Configuration with BigOption 16

7.3.1 Active State Management 17

7.4 Big Constant for Common Use - BigZero, BigOne, BigTwo, BigThree and BigPi 18

7.5 Performance vs Not Responding with Yield 18

7.6 Calculation Interruption & Cancellation Support 19

8 The Automated Test Suite: Architecture and Execution 21

8.1 What a 239/239 All-Green Pass Signifies 21

8.2 How to run the Test Suite 22

9 Conclusion 22

10 Appendix A – Test Suite Results 22

List of Tables

Table 7-1: BigDecimal Methods 11
Table 7-2: BigDecimal Methods Continued 12
Table 7-3: BigDecimal Properties 13
Table 7-4: .ValueEx Property VBA Types 14
Table 7-5: BigOption Properties 17
Table 7-6: Yield Class Methods 19
Table 7-7: Yield Class Property 19

List of Figures

Figure 3-1: The BigCalculator 6
Figure 4-1: Excel Add-ins Folder 7
Figure 4-2: Unblock Downloaded BigDecimal Add-in 7
Figure 5-1: Adding BigDecimal_v1_6 as a Reference 8
Figure 7-1: Interrupting Execution with No Error Handler 20

1 Introduction

Welcome to **BigDecimal**, a a high-performance, arbitrary-precision decimal library designed to break the 15-digit precision barrier of standard Excel calculations. Heavily inspired by the robust BigDecimal implementations found in enterprise languages like Java, this add-in brings industrial-grade numerical accuracy directly to your spreadsheets.

Whether you are performing complex financial reconciliations or deep scientific research, BigDecimal ensures that your results are never compromised by floating-point errors or rounding limitations.

1.1 Why BigDecimal?

Standard Excel uses the IEEE 754 floating-point standard, which is excellent for speed but can introduce "silent" errors in very large or very small numbers. BigDecimal for Excel replaces this with an exact-value representation, allowing you to define the exact scale and precision required for your specific use case.

2 Key Features and Capabilities

- **Breaks the 15-Digit Excel Barrier:** Replaces standard IEEE 754 floating-point calculations with an exact-value engine, allowing you to compute across dozens or hundreds of digits without floating-point errors.
- **Desktop-Grade "BigCalculator" Interface:** Includes a fully functional display application to run hyper-precise formulas, advanced algebraic operations, and instant high-scale mathematical constants (π and *e*) on the fly.
- **Dynamic Precision Control (Scale):** Allows you to dynamically expand or contract the decimal scale (defaulting to 100 decimal places) at any point during a calculation.
- **Auto-Casting & Smart Formatting:** Implicitly converts native VBA data types and strings into high-precision objects mid-calculation, while automatically stripping out non-significant zeros for clean string outputs.
- **Comprehensive Advanced Math Library:** Includes direct methods for complex transcendental operations (x^y, $\sqrt{x}$, $\sqrt[3]{x}$, $n!$), natural and base-10 logarithms (ln, log), natural exponents (e^x), as well as absolute values and inversion.
- **Deep Numeric Component Extraction:** Features specialized parsing functions like Trunc (to drop decimals without rounding), Fract (to isolate only the fractional component), and standard bounding functions (Floor and Ceiling).
- **Advanced Number Theory Utilities:** Includes high-precision engines for calculating the Greatest Common Divisor (GCD) and the Least Common Multiple (LCM) of massive numbers.
- **Flexible Scale Operations:** Built-in array-shifting mechanisms (Shift) to seamlessly move decimal points left or right, alongside explicit Round and Trim utility methods.

- **Full Suite of Precision Logic Gates:** Provides strict, character-for-character comparison operators (IsEq, IsNEq, IsGT, IsLT, IsGE, IsLE) and sign checks (IsZero, IsNeg, IsPos) to safely bypass standard 64-bit hardware limits.
- **Global Configuration Tracking (BigOption):** Uses a live management registry to instantly propagate scale, rounding, and debugging changes across every active object in memory simultaneously.
- **Memory-Optimized Singletons:** Employs pre-loaded, immutable constants (BigZero, BigOne, BigTwo) that bypass global tracking by BigOption to prevent memory overhead during intense calculation loops.
- **Anti-Freeze & Multi-Tasking Safeguards (Yield):** Prevents Excel from falling into a "Not Responding" state by periodically triggering DoEvents instructions at customizable loop intervals.
- **Hardware-Level Calculation Abort:** Monitors the keyboard directly via Windows API, allowing users to safely halt heavy calculations at any time by pressing the **ESC** key without risking raw code exposure or system crashes.
- **130-Point Micro-Assertion Test Suite:** Uses a built-in automated test harness that programmatically injects volatile, extreme inputs (asymmetric scales, infinite repeating patterns, multiple trailing zeros) to stress-test every layer of the mathematical kernel.
- **Character-for-Character Test Validation:** Intercepts internal array data, extracts string representations via .ValueEx, and explicitly validates test results against high-fidelity mathematical benchmarks to instantly expose any precision leaks or broken dependencies.

3 The BigCalculator: Desktop-Grade Precision in Excel

Serving as the flagship showcase for the **BigDecimal** add-in, the **BigCalculator** (Figure 3-1) is a fully functional, real-time application designed to demonstrate the sheer speed and raw numerical depth of the underlying mathematical kernel. Boasting a large, highly legible display, it provides a user-friendly interface for executing hyper-precise calculations that easily transcend the limitations of traditional 64-bit hardware.

The calculator exposes a comprehensive suite of advanced mathematical tools directly on its control panel: standard operations (+, −, ×, ÷), algebraic controls (Absolute Value, Sign Inversion, Truncation, Fraction Splitting, Modulo, and Strict Comparisons), alongside rapid transcendental engines like powers (x^y), Natural and Base-10 Logarithms (*ln*, *log*), Exponentials (*exp*), Square Roots (√), Factorials ($n!$), and instant, arbitrary-scale generations of mathematical constants like π and e.

To give developers absolute control over the execution layout, the application features an integrated Precision **Scale** button and an editable configuration box. Users can dynamically input their exact target decimal scale right on the interface, instantly recalibrating the calculator to process complex formulas across dozens or hundreds of digits with absolute,

character-for-character fidelity. As the official example workbook, the **BigCalculator** stands as concrete proof that Excel can be transformed into a desktop-grade, infinite-precision computing station.

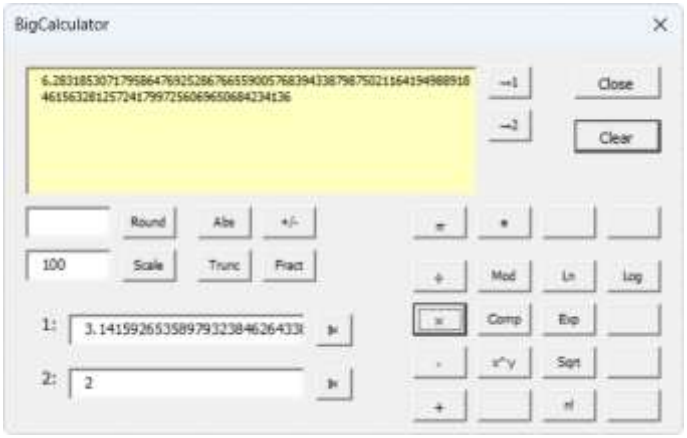


Figure 3-1: The BigCalculator

4 Installation of BigDecimal v1.6.xlam

Follow the steps below to install the **BigDecimal** add-in and integrate it with Microsoft Excel. These steps ensure the library is correctly registered and available for use within the VBA Editor.

4.1.1 Downloading the BigDecimal Add-in

To install the Add-in, download the **BigDecimal v1.6.xlam** file from the GitHub repository and move it to the local Microsoft Add-ins directory:

C:\Users\[*YourUsername*]\AppData\Roaming\Microsoft\AddIns

Note: Replace [YourUsername] with your active Windows login name.

Quick Access Shortcut:
Alternatively, you can access this folder directly by pasting the following environment variable into the File Explorer address bar:

%appdata%\Microsoft\AddIns

Once the folder is open, drag and drop the BigDecimal v1.6.xlam file from your **Downloads** folder into the **Addins** folder as in Figure 4-1.

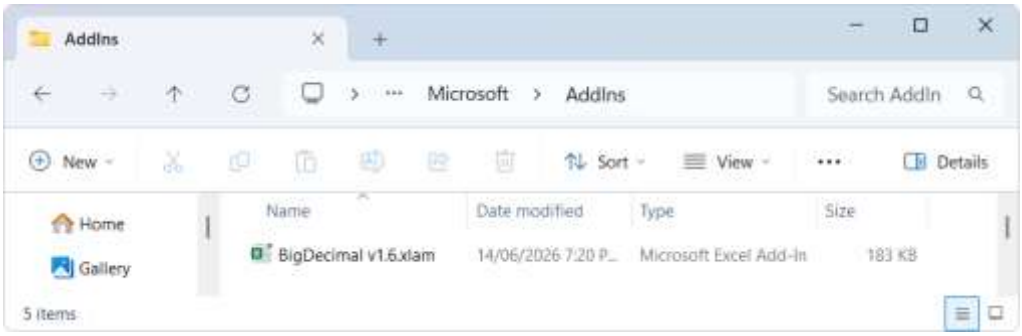


Figure 4-1: Excel Add-ins Folder

4.1.2 Important: Unblocking the Add-in

Because the file was downloaded from an external source (GitHub), Windows will apply a 'Mark of the Web' security attribute, which prevents the Add-in from executing macros. Even if the file is placed in the correct directory, it will not function until it is manually unblocked.

To unblock the Add-in:

1. Navigate to the %appdata%\Microsoft\AddIns folder
2. **Right-click** the BigDecimal v1.6.xlam file.
3. Select **Properties**.
4. At the bottom of the **General** tab, look for **Security**.
5. Check the **Unblock** box and click **Apply**.

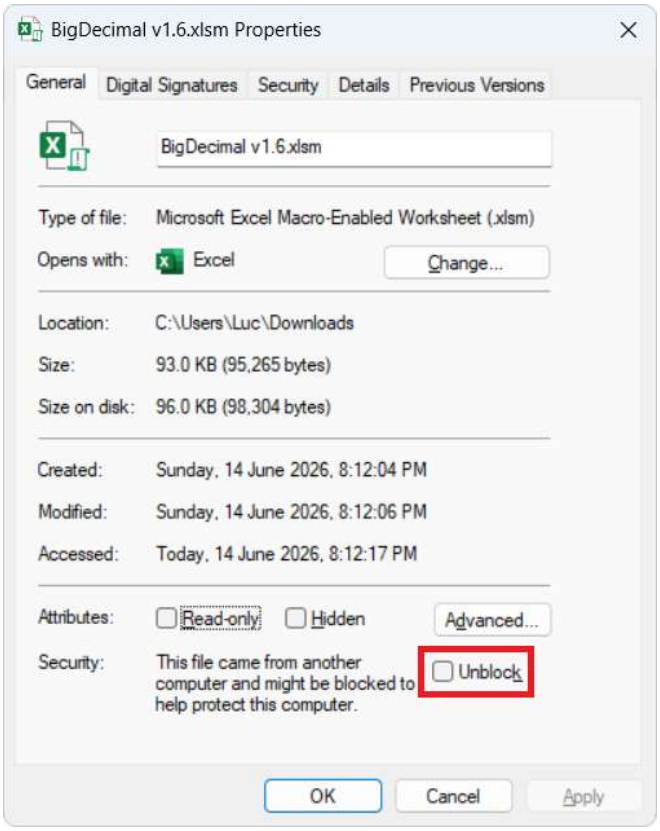


Figure 4-2: Unblock Downloaded BigDecimal Add-in

5 Working with BigDecimal

Before the **BigDecimal** class can be utilized in your VBA project, it must be properly initialized within the VBA environment. This involves linking the Add-in to your specific workbook and ensuring the VBA compiler can access the library's properties and methods.

5.1 Referencing BigDecimal in VBA

Once the Add-in is installed and unblocked, you must establish a reference to the library within your specific VBA Project. This allows the VBA compiler to recognize the **BigDecimal** class and provide IntelliSense support while coding.

Follow these steps to enable the reference:

1. Open Excel.
2. Go to the **Developer** tab (If you don't see it, right-click any ribbon tab > Customize the Ribbon > Check "Developer").
3. Click **View Code**.
4. Go to the **Tools** menu > **References..** and locate BigDecimal_v1_6 and select it as in Figure 5-1. A tick mark will appear next to BigDecimal_v1_6.
5. Click **OK**.

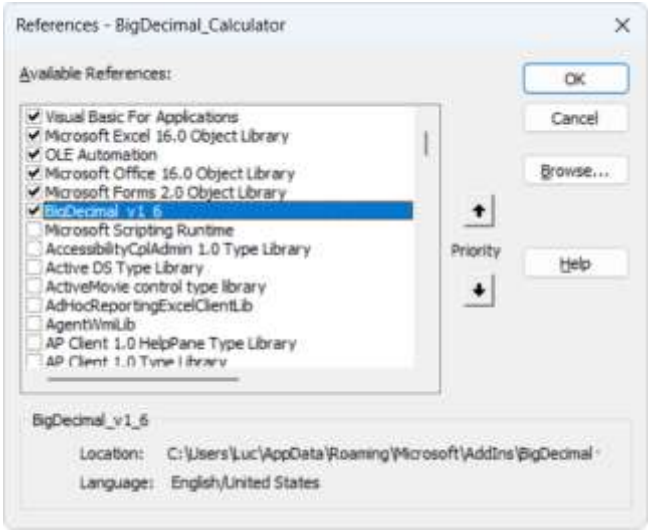


Figure 5-1: Adding BigDecimal_v1_6 as a Reference

At this stage the add-in is now available to use in your VBA Project.

6 BigDecimal "Quick Start" Guide

BigDecimal is designed to work seamlessly within the Excel environment, its implementation respects the standard VBA architectural patterns used by Microsoft Office. To ensure maximum compatibility and performance across different workbooks, the library utilizes a

controlled instantiation model. This approach prevents common reference errors and ensures that every BigDecimal object is correctly initialized with the high-precision engine.

6.1 Instantiating the BigDecimal Class

In standard object-oriented programming, you would typically instantiate a class using the `New` keyword. However, due to the **`PublicNotCreatable`** nature of classes within an Excel Add-in environment, the BigDecimal class cannot be created directly by external projects.

To bypass this limitation, the Add-in provides a global factory function: `New_BigDecimal()`. This function serves as the entry point for object creation, allowing you to initialize a new BigDecimal instance. It optionally accepts a string parameter to set the initial value of the number, ensuring precision is maintained from the moment of instantiation.

```
Public Function New_BigDecimal(Optional StrValue As String = "") As
BigDecimal
```

6.2 Casting standard types

To simplify workflows, the Add-in includes the CBgDec() casting function. This function mirrors native VBA conversion functions (like CDbl or CDec), accepting standard data types and returning a fully initialized BigDecimal object.

```
Public Function CBgDec(Value As Variant) As BigDecimal
```

Supported Conversions

The CBgDec() function safely handles the following native types by converting them to their string representations internally before initializing the BigDecimal object:

- **Numeric Types:** Integer, Long, LongLong, Single, Double.
- **Strings:** Standard text representations of numbers (e.g., "123.4567890123456789").

6.3 Implementation Example: Basic Addition

The following example, adapted from the **BigCalculator** source code, demonstrates how to instantiate two BigDecimal objects from user input, perform basic validation, and output the result of an addition operation.

```
Private Sub Add_Click()
    Dim Num1 As BigDecimal
    Dim Num2 As BigDecimal

    With BigDecimalForm
        ' Instantiate objects using the New_BigDecimal factory function
        Set Num1 = New_BigDecimal(.Num1.Value)
        Set Num2 = New_BigDecimal(.Num2.Value)

        ' Validate inputs and check for successful instantiation
        If .Num1.Value = "" Then
            ErrBox "Number 1 is empty!", "Input Error"
            Exit Sub
        ElseIf .Num2.Value = "" Then
            ErrBox "Number 2 is empty!", "Input Error"
            Exit Sub
        ElseIf Num1 Is Nothing Then
            ErrBox "Number 1 is invalid!", "Validation Error"
            Exit Sub
        ElseIf Num2 Is Nothing Then
            ErrBox "Number 2 is invalid!", "Validation Error"
            Exit Sub
        End If

        ' Perform addition and return the result as a string
        .Answer.Value = Num1.Plus(Num2).ValueEx
    End With
End Sub
```

6.4 Language Limitations: Operator Overloading

A notable constraint of the VBA environment is the lack of **Operator Overloading**. In many modern languages, you can redefine standard operators (such as +, −, ×, ÷) to work directly with custom classes. Ideally, one would use a syntax like Set Result = Num1 + Num2; however, this is not supported in VBA.

Consequently, there is no direct workaround for this language limitation. All arithmetic operations must be performed by calling the explicit methods of the BigDecimal class. While this requires a slightly more verbose coding style, the method names are intuitive and follow standard mathematical naming conventions.

7 Functional Specifications

This section provides a detailed technical breakdown of the public classes, methods, and properties that comprise the **BigDecimal** framework. These components form the core engine of the library, enabling the programmatic initialization of high-precision variables, the execution of complex mathematical operations, and the fine-tuning of calculation parameters such as scale and rounding. By leveraging these members, developers can extend the numerical capabilities of their Excel applications far beyond the standard limits of floating-point arithmetic.

7.1 BigDecimal Methods

The following table outlines the comprehensive suite of arithmetic and utility methods supported by the **BigDecimal** class, designed to provide consistent results for high-precision computational tasks. **`Num1`** and **`Num2`** are BigDecimals. **`Place`** is a Long and supports positive and negative values. **`Scale`** is a Long and supports positive values only.

Methods	VBA Syntax (BigDecimal)	Notes
Plus	`Num1.Plus(Num2)`	Returns the sum of Num1 and Num2 (+).
Minus	`Num1.Minus(Num2)`	Returns the difference of Num1 and Num2 (−).
Times	`Num1.Times(Num2)`	Returns the product of Num1 and Num2 (×).
Divide	`Num1.Divide(Num2)`	Returns the quotient of Num1 and Num2 (÷).
Remdr	`Num1.Remdr(Num2)`	Returns the remainder of the division.
Pct	`Num1.Pct(Num2)`	Calculate the percentage (%).
Sqr	`Num1.Sqr()`	Calculate the square (x^2).
Sqrt	`Num1.Sqrt()`	Calculates the square root to current precision ($^2\sqrt{}$).
Cube	`Num1.Cube()`	Calculate the cube (x^3).
Cubert	`Num1.Cubert()`	Calculate the cube root ($^3\sqrt{}$).
Pow	`Num1.Pow(Num2)`	Calculates the power (x^y).
Pow10	`Num1.Pow10()`	Calculate 10 to the power (10^x).
Log	`Num1.Log()`	Calculate logarithm to the base 10 (log_{10})
Ln	`Num1.Ln()`	Calculates the natural logarithm (*ln*).
Exp	`Num1.Exp()`	Calculates the natural exponent (e^x).
Invert	`Num1.Invert()`	Calculate the inverse.
Fact	`Num1.Fact()`	Calculates the product of all positive integers (*x!*).
AbsVal	`Num1.AbsVal()`	Returns the absolute (non-negative) value.
Neg	`Num1.Neg()`	Returns the value multiplied by -1.
Fract	`Num1.Fract()`	Returns only the decimal/fractional part.
Trunc	`Num1.Trunc(Place)`	Truncates digits left or right of decimal point.
Floor	`Num1.Floor(Place)`	Rounds down to specified left/right increment.
Ceiling	`Num1.Ceiling(Place)`	Rounds up to specified left/right increment.
Shift	`Num1.Shift(Place)`	Shift the decimal point to the left or right.
Round	`Num1.Round(Scale)`	Rounds to a specified number of decimal places.
Trim	`Num1.Trim()`	Trim leading and trailing 0.

Table 7-1: BigDecimal Methods

Methods	VBA Syntax (BigDecimal)	Notes
ToRad	`Num1.ToRad()`	Convert Degrees to Radians.
ToDeg	`Num1.ToDeg()`	Convert Radians to Degrees.
Sin	`Num1.Sin()`	Returns the sine of a radian angle.
Cos	`Num1.Cos()`	Returns the cosine of a radian angle.
Tan	`Num1.Tan()`	Returns the tangent of a radian angle.
Sinh	`Num1.Sinh()`	Returns the hyperbolic sine of radians.
Cosh	`Num1.Cosh()`	Returns the hyperbolic cosine of radians.
Tanh	`Num1.Tanh()`	Returns the hyperbolic tangent of radians.
Sec	`Num1. Sec()`	Returns the secant of a radian angle.
Csc	`Num1.Csc()`	Returns the cosecant of a radian angle.
Cot	`Num1.Cot()`	Returns the cotangent of a radian angle.
GCD	`Num1.GCD(Num2)`	Largest number that divides both with no reminder.
LCM	`Num1.LCM(Num2)`	Smallest that is perfect multiple of both.
Min	`Num1.Min(Num2,…)`	Return the smallest decimal from the list.
Max	`Num1.Max(Num2,…)`	Return the largest decimal from the list.
IsZero	`Num1.IsZero()`	True when Num1 is equal to zero (=0).
IsNeg	`Num1.IsNeg()`	True when Num1 is negative (<0).
IsPos	`Num1.IsPos()`	True when Num1 is positive (>0).
IsEq	`Num1.IsEq(Num2)`	True when Num1 is equal to (=) Num2.
IsNEq	`Num1.IsNEq(Num2)`	True when Num1 is not equal to (≠) Num2.
IsGT	`Num1.IsGT(Num2)`	True when Num1 is greater than (>) Num2.
IsLT	`Num1.IsLT(Num2)`	True when Num1 is less than (<) Num2.
IsGE	`Num1.IsGE(Num2)`	True when Num1 is greater than or equal to (≥) Num2.
IsLE	`Num1.IsLE(Num2)`	True when Num1 is less than or equal to (≤) Num2.
CmpTo	`Num1.CmpTo(Num2)`	Returns -1 (less), 0 (equal), or 1 (greater).

Table 7-2: BigDecimal Methods Continued

7.2 BigDecimal Properties

The **BigDecimal** class includes a set of public properties that provide critical insights into the state and value of your numerical objects. Unlike standard numeric types in VBA, which require complex parsing to analyse, these properties allow you to directly access the internal characteristics of a number—such as its string representation, and its current scale (the number of digits after the decimal point). By utilizing these properties, developers can easily

integrate high-precision results into the Excel UI or use them as conditional logic in complex financial and scientific algorithms.

Property	Access	Notes
.Value	Read/Write	Gets or sets the BigDecimal object itself. This is the default property called when no property is specified and can be a `vbInteger`, `vbLong`, `vbLongLong`, `vbSingle`, `vbDouble`, `vbString`.
.ValueEx	Read Only	Gets the value cast to a `vbInteger`, `vbLong`, `vbLongLong`, `vbSingle`, `vbDouble`, `vbString`. The default is `vbString` when no parameter is specified.
.TarScale	Read/Write	Gets or sets the number of decimal places for a BigDecimal number. Setting this can truncate or pad the number. Default is 100 decimal places.
.AutoRound	Read/Write	A boolean flag that determines if the BigDecimal number should be automatically rounded. Default is True.
.OutputDebug	Read/Write	Enables internal logging to the VBA Immediate Window (Ctrl+G). Default is False.

Table 7-3: BigDecimal Properties

7.2.1 The Default .Value Property

The `.Value` property is configured as the **Default Member** of the BigDecimal class. This means the property is automatically invoked when the object is referenced without an explicit property call. This design pattern allows for cleaner, more intuitive code that mimics the behaviour of native VBA variables.

```
' Instead of explicit property calls:
Num1.Value = Num2.Value

' You can simply write:
Num1 = Num2
```

In the example above, the default property mechanism handles the transfer of data: `Num2` yields its value, and `Num1` accepts it. This syntactic sugar is particularly powerful when passing BigDecimal objects into standard Excel functions or performing quick assignments within loops.

7.2.2 The .ValueEx Property

While the default `.Value` property is ideal for general assignments, the `.ValueEx` property provides a typed interface for extracting data. This property accepts a single parameter—a standard VBA `VbVarType` constant—to determine the format of the return value. When no parameter is specified, the default type is `vbString`.

Syntax

```
result = BigDecimal.ValueEx(ReturnType)
```

Supported Return Types

Constant	VBA Type	Use Case
vbInteger	Integer	Small whole numbers (-32,768 to 32,767).
vbLong	Long	Standard 32-bit integer operations.
vbLongLong	LongLong	64-bit integer operations (Available in 64-bit Office).
vbSingle	Single	Low-precision floating point.
vbDouble	Double	Double-precision floating point.
vbString	String	Standard UTF-16 character string. This is the default when no ReturnType is specified.

Table 7-4: .ValueEx Property VBA Types

Example Usage

```
Dim MyStr As String
Dim MyDbl As Double

' Retrieve as a string to preserve every decimal place
MyStr = Num1.ValueEx

' Retrieve as a double for use in standard VBA math
MyDbl = Num1.ValueEx(vbDouble)
```

7.2.3 The .TarScale Property

The `.TarScale` property determines the maximum number of digits allowed after the decimal point. This property is crucial for managing computational overhead and ensuring that operations like division—which can result in infinite repeating decimals—have a defined termination point.

Unlike standard VBA types that have a fixed precision, BigDecimal allows you to expand or contract this scale dynamically.

Usage and Behavior

- **Expansion:** Increasing `.TarScale` will effectively "pad" the number with trailing zeros, preparing it for higher-precision arithmetic.
- **Contraction:** Decreasing `.TarScale` will trigger a precision adjustment. Depending on the internal logic, the number will be truncated or rounded to fit the new scale.
- **Read/Write:** This property can be checked to verify current precision or set to enforce a specific scale before a calculation.

Example: Controlling Precision

```
Dim Num1 As New BigDecimal
Num1 = "10.55555"

' Set the scale to 2 decimal places
Num1.TarScale = 2

Debug.Print Num1.ValueEx   ' Output: “10.55” (truncation)

' Increase scale for scientific precision
Num1.TarScale = 10
Debug.Print Num1.ValueEx   ' Output: “10.55”
```

Performance Tip: While BigDecimal can handle very high scales, setting an unnecessarily large `.TarScale` (e.g., 1,000+) for simple calculations may impact performance. It is best practice to set the scale to the minimum required for your specific use case.

7.2.4 Auto-Casting of Standard Types

To further enhance usability, the BigDecimal class supports **Auto-Casting** (Implicit Conversion). When a standard VBA numeric type or string is passed to a BigDecimal method (see Table 7-3) or assignment, the class automatically converts the input into a high-precision object before processing.

This eliminates the need to manually wrap literal values or standard variables, making arithmetic operations feel natural and concise.

Supported Types for Auto-Casting:

- **Integers:** `vbInteger`, `vbLong`, `vbLongLong`
- **Floating Point:** `vbSingle`, `vbDouble`
- **Strings:** `vbString` (Parsed directly into the internal format)

Example: Simplified Arithmetic

```
' Without auto-casting, you would have to write:
' Num1 = Num1.Add(New_BigDecimal("1"))

' With auto-casting, standard types are handled instantly:
Num1 = Num1.Add(1)          ' Long is auto-cast
Num1 = Num1.Multiply(0.5)   ' Double is auto-cast
Num1 = "100.50"     ' String is auto-cast via the default .Value property
```

Developer Note: Auto-casting ensures that even when interacting with standard VBA types, the precision of the existing BigDecimal object remains the priority, preventing accidental "precision leakage" from the standard Double type until the operation is complete.

Precision Warning: When using `.ValueEx(vbDouble)` or `.ValueEx(vbSingle)`, values may be rounded or truncated to fit the limitations of the native VBA type. For values

exceeding the range of a Long or Double, an overflow error may occur unless handled within your implementation.

7.2.5 Effortless Precision with Auto-Trimming

The **BigDecimal** class is designed to deliver clean, professional output without the need for manual string manipulation. While the internal object may retain leading and trailing zeros to preserve the integrity of its data structure, the **`.ValueEx`** property handles the cleanup for you. Every time you access the value via this property, the library automatically trims any non-significant zeros, ensuring your results are presented in their most concise and readable form while maintaining the high-precision accuracy you require.

7.2.6 Mutable vs. Immutable Properties

Most properties in the **BigDecimal** class are **Mutable**, meaning you can change the value of an existing object without needing to create a new one using the factory function.

- **Direct Modification:** By setting `.Value`, you can reuse an existing BigDecimal variable and update its internal data instantly.
- **Precision Control:** Manually setting the .TarScale allows you to force a specific decimal length on an existing number, which is useful for formatting financial data (e.g., forcing 2 decimal places).
- **The Pi Exception:** Unlike the other properties, **Pi** is a constant. It cannot be overwritten, as it represents a fundamental mathematical value. Attempting to set Num1.Pi = 3 will result in a compilation error, ensuring the integrity of your geometric calculations.

7.3 Global Configuration with BigOption

The **BigOption** class provides a centralized mechanism for managing the behavior of all BigDecimal objects within your application. By setting properties on this global object, you define the default environment for any new BigDecimal instantiated thereafter.

This "seamless management" approach ensures that you don't have to manually configure every variable, reducing boilerplate code and preventing precision mismatches in large-scale projects.

Supported Global Options

Property	Description
.TarScale	Sets the default number of decimal places for all newly created BigDecimal objects. Default is 100.
.AutoRound	A boolean flag that determines if numbers should be automatically rounded (e.g., Round Half-Up) or truncated when they exceed the TarScale. Default is True.
.AddTrailingZeros	A boolean flag that determines if a number is padded with zeros during a .Round operation. This flag requires the AutoRound flag to be True before a number will be padded with zeros.
.OutputDebug	Enables internal logging to the VBA Immediate Window (Ctrl+G). Useful for tracing complex calculation steps or identifying performance bottlenecks. Default is False.
.Cache	A Boolean flag to switch the cache on or off for the logarithm and Pi functions. The Cache is set to True by default.

Table 7-5: BigOption Properties

There are no methods exposed for the BigOption class.

7.3.1 Active State Management

The **BigOption** class is not merely a collection of defaults; it is a **Live Manager**. Using an internal dictionary, it maintains a registry of every active BigDecimal instance within the project. This allows for dynamic, real-time updates across the entire application state.

Key Mechanisms:

- **Global Synchronization:** When you change a property on BigOption (such as `.TarScale`), the change is instantly propagated to every existing BigDecimal object currently in memory.
- **Lifecycle Awareness:** The manager intelligently tracks the lifecycle of your objects. When a BigDecimal is terminated or set to Nothing, BigOption removes it from the registry to prevent memory leaks and ensure that only valid, active objects receive updates.
- **Real-Time Control:** This architecture allows you to change the behaviour of your algorithms on the fly. For example, you can increase the precision of all active variables right before a sensitive calculation and lower it immediately afterward to optimize performance.

Example: Real-Time Global Updates

```
Dim Num1 As New BigDecimal, Num2 As New BigDecimal
Num1 = 1.12345
Num2 = 9.87654

' Both Num1 and Num2 are currently at default TarScale (e.g., 20)
Debug.Print Num1.ValueEx  ' Outputs 1.123450...

' Update every active BigDecimal in the project instantly
BigOption.TarScale = 2

Debug.Print Num1.ValueEx  ' Now outputs 1.12
Debug.Print Num2.ValueEx  ' Now outputs 9.87
```

Advanced Note: Because BigOption manages the collection via a dictionary, the "Auto-Casting" and "Default Value" features remain lightning-fast. The manager ensures that the overhead of tracking objects does not interfere with the high-speed arithmetic processing of the individual classes.

7.4 Big Constant for Common Use - BigZero, BigOne, BigTwo, BigThree and BigPi

To minimize memory allocation overhead and eliminate repetitive object instantiation during intensive arithmetic calculations, the library exposes four global singleton properties: **`BigZero`**, **`BigOne`**, **`BigTwo`** and **`BigThree`**. Implemented within a standard module using localized Static object references, these read-only properties enforce a lazy-loading design pattern that instantiates each constant exactly once for the duration of the application lifecycle. Upon initial access, each instance invokes an explicit **`.Unregister`** directive to detach itself from the global configuration tracking system (**BigOption**). This crucial isolation layer ensures that these fundamental mathematical primitives remain entirely immutable, protecting their state from accidental pollution or modification by any ambient global scaling adjustments made during downstream operations.

Conversely, the **BigPi** (π) property functions as a dynamic, scale-aware generator rather than a rigid structural singleton. To optimize performance, BigPi leverages a built-in high-precision cache; if the requested scale falls within this default threshold, it instantly delivers a pre-computed result. However, when ambient precision boundaries (TarScale) exceed this cached limit, the property bypasses the buffer to actively recalculate a fresh, fully compliant BigDecimal instance on the fly, ensuring it dynamically honors the exact environmental configurations defined at the moment of execution.

7.5 Performance vs Not Responding with Yield

The BigDecimal **Yield** class provides an essential cooperative multitasking mechanism designed to preserve application stability during intensive, high-precision mathematical operations. Because arbitrary-precision arithmetic can require deep, multi-layered processing loops across the core BigDecimal and BigOption modules, execution can easily monopolize the **single-threaded** Excel interface, triggering a frustrating "Not Responding" status in the window caption. To counter this, the `.Events` method is called directly within the execution

loops to increment the internal counter. Once this counter hits a predefined threshold, the method intercepts the cycle and fires a native VBA `DoEvents` instruction, allowing Excel to safely refresh its UI and process pending operating system messages.

Empirically, a default threshold of 1000 iterations establishes an optimal performance equilibrium—frequent enough to keep the interface responsive, yet sparse enough to avoid the execution overhead that comes with over-polling the operating system. To ensure complete developer control, the class exposes a `.MaxCount` property allowing this 1000-iteration benchmark to be dynamically tailored to the host machine's processing power, alongside a `.Clear` method to seamlessly reset the loop tracking metrics back to zero before starting a new calculation chain.

Method	Access	Notes
.Events	Read/Write	Used inside loops to increment the internal counter. When the threshold is reached the DoEvents instruction is triggered.
.Clear	Read/Write	Called to clear the internal counter.

Table 7-6: Yield Class Methods

Property	Access	Notes
.MaxCount	Read/Write	Gets or sets the default loop iteration threshold before DoEvents is triggered. The default is 1000.

Table 7-7: Yield Class Property

7.6 Calculation Interruption & Cancellation Support

To maintain application responsiveness during long-running or resource-intensive operations, the **BigDecimal** library includes an integrated, low-overhead interruption manager via the **Yield.Events** utility. The engine monitors hardware states directly using the Windows API (**GetAsyncKeyState**), throttling checks to once every 1000 loop iterations to ensure virtually zero performance degradation. Users can safely pause and cancel an active calculation at any time by pressing the ESC key. Confirming the cancellation prompt gracefully halts the entire execution pipeline by raising a distinct, bubble-up custom runtime error (**vbObjectError + 513**). This design prevents application freezing, eliminates the risk of raw code exposure from standard VBA macro breaks, and allows parent procedures to capture the abort event cleanly via standard error handling for a safe, managed rollback.

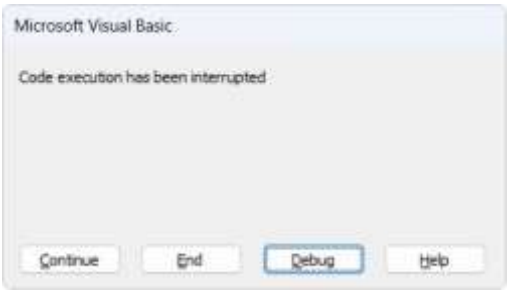


Figure 7-1: Interrupting Execution with No Error Handler

In the absence of localized error-handling routines, an interruption via the ESC key will trigger the standard VBA runtime environment dialog (as shown in Figure 7-1), exposing the native "Continue, End, or Debug" options. Selecting "End" will abruptly terminate the current execution thread. To achieve a more seamless and professional user experience, implementing structured error-trapping is highly recommended. This approach isolates the custom runtime error natively, enabling parent architectures to manage the cancellation cleanly, as demonstrated in the master pipeline example below:

Example master pipeline with centralised interruption handling:

```
Public Sub RunBigMathProcess()
    ' 1. Activate the error router
    On Error GoTo ErrorHandler

    ' 2. Optimize Excel for speed
    Application.ScreenUpdating = False
    Application.Calculation = xlCalculationManual

    Dim A As New BigDecimal, B As New BigDecimal, Result As BigDecimal
    A = "1234567890123456"
    B = "987654321"

    MsgBox "Starting massive calculation. Press ESC at any time to abort.",
vbInformation, "Process Started"

    ' 3. Execute library function (which calls Yield.Events internally)
    Set Result = A.GCD(B)

    ' 4. If we get here, the calculation completed without an abort!
    MsgBox "Calculation Complete! GCD is: " & Result.ValueEx,
vbInformation, "Success"

CleanUp:
    ' 5. CRITICAL ZONE: This block ALWAYS runs, even if an error or abort
occurs.
    ' Always restore environmental properties here so Excel doesn't stay
frozen.
    Application.ScreenUpdating = True
    Application.Calculation = xlCalculationAutomatic
    Exit Sub

ErrorHandler:
    ' 6. Identify the specific error code raised by the Yield utility
    If Err.Number = vbObjectError + 513 Then
```

```
            ' Graceful exit message for an explicit user cancellation
            MsgBox "The calculation was cancelled by the user. No data was
modified.", vbExclamation, "Operation Aborted"
        Else
            ' Fallback to handle any unexpected program errors gracefully
            MsgBox "An unexpected system error occurred:" & vbCrLf & _
                   "Error #" & Err.Number & ": " & Err.Description, vbCritical,
"Execution Error"
        End If

        ' 7. Send the execution flow down to the CleanUp block to safely exit
        Resume CleanUp
End Sub
```

8 The Automated Test Suite: Architecture and Execution

The automated test suite serves as the structural scaffolding for the BigDecimal library, acting as a rigorous, real-time quality control system. Implemented directly within the VBA environment, the suite operates by executing a sequential matrix of 130 distinct micro-assertions that isolate and stress every layer of the mathematical kernel.

It works by programmatically injecting highly volatile string inputs—such as extreme asymmetric scales, multiple leading or trailing zeros, explicit signs, and infinite repeating patterns—directly into the parser. The engine then processes these values through its underlying array-manipulation routines.

Upon completion of each operation, the test suite intercepts the internal high-precision array data, extracts the visible representation via a string casting method (`.ValueEx`), and performs a strict, character-by-character validation against verified, high-fidelity mathematical benchmarks.

By systematically running these checks across all primary operations (+, −, ×, ÷), sign states, logic gates, and complex transcendentals, the test harness instantly exposes any structural degradation, precision leaks, or broken internal dependencies across the entire code base.

8.1 What a 239/239 All-Green Pass Signifies

Achieving a flawless **239 / 239 "All Green"** pass rate shows the BigDecimal library is a robust, production-grade mathematical engine. In arbitrary-precision development, adding advanced layers like fractional exponentiation (`.Pow`) or long division (`.Divide`) frequently introduces subtle regression bugs that silently break core routines like addition and subtraction. Surviving this aggressive matrix shows that the entire library is perfectly integrated and structurally sound.

From an application standpoint, this means the engine is ready for deployment:

- **Financial Readiness:** Standard programming types suffer from the native IEEE 754 floating-point limitation, which truncates data at roughly 15 to 16 digits. In global enterprise environments or compound financial models, these tiny, hidden truncation errors roll up into massive balance discrepancies over millions of iterations. This library ensures absolute transactional integrity by executing ripple-rounding carries and zero-annihilations.
- **Scientific Readiness:** For high-fidelity simulations, engineering models, or statistical computations, the engine maintains character-for-character fidelity with absolute mathematical reality out to dozens of arbitrary decimal scales.

Ultimately, this complete pass marks the library's evolution from an experimental programming project into a bulletproof, institutional-grade mathematical kernel.

8.2 How to run the Test Suite

The comprehensive test suite is integrated directly into the **BigCalculator** spreadsheet. To validate the engine, simply execute the `RunAllTests and RunAllTests2` macros, which streams live verification outputs to the VBA **Immediate Window**. For a complete log of the pre-release deployment run, please refer to Appendix A – Test Suite Results for the official benchmark log recorded prior to production release.

9 Conclusion

Thank you for choosing **BigDecimal**.

The goal of this library is to provide a robust, high-precision numerical engine that bridges the gap between standard VBA data types and the rigorous accuracy required for modern financial and scientific applications.

I hope the current feature set—from the intelligent **auto-trimming** logic to the definitive precision control of **TarScale**—empowers you to build more reliable and professional tools. As the development of this class continues, I am committed to maintaining this balance of mathematical integrity and developer-friendly flexibility.

10 Appendix A – Test Suite Results

The following log represents the official baseline verification run of the BigDecimal mathematical kernel, executed in the VBA Immediate Window immediately prior to the production release.

```
======= STARTING AUTOMATED TESTS ================================

 [+]   1 Basic Addition ..................................... [PASS]
 [+]   2 Addition with Carry-Over ........................... [PASS]
 [+]   3 Positive and Negative Addition ..................... [PASS]
 [+]   4 Negative and Positive Addition ..................... [PASS]
 [+]   5 Negative and Negative Addition ..................... [PASS]
 [+]   6 Massive Asymmetric Addition ........................ [PASS]
 [+]   7 Perfect Zero Annihilation Addition ................. [PASS]
 [+]   8 Cascading Ripple Borrow ............................ [PASS]
 [+]   9 Sign Reversal Subtraction .......................... [PASS]
 [+]  10 High-Precision Sign Reversal ...................... [PASS]
 [+]  11 Double Negative Addition Route .................... [PASS]
 [+]  12 Subtracting From Negative ......................... [PASS]
 [+]  13 Perfect Zero Annihilation Subtraction ............. [PASS]
 [+]  14 Scale Alignment Multiplication .................... [PASS]
 [+]  15 Cascading Carry Multiplication .................... [PASS]
 [+]  16 Negative Multiplication by Zero ................... [PASS]
 [+]  17 Double Negative Multiplication .................... [PASS]
 [+]  18 Mixed Sign Multiplication ......................... [PASS]
 [+]  19 Precision Expansion Scale Sum ..................... [PASS]
 [+]  20 Micro-Decimal Multiplication ...................... [PASS]
 [+]  21 Large Integer Square Multiplication ............... [PASS]
 [+]  22 Exact Integer Division ............................ [PASS]
 [+]  23 Terminating Decimal Division ...................... [PASS]
 [+]  24 Repeating Decimal Division ........................ [PASS]
 [+]  25 Repeating Decimal Loop (22/7) ..................... [PASS]
 [+]  26 Small Divided by Large ............................ [PASS]
 [+]  27 Positive Divided By Negative ...................... [PASS]
 [+]  28 Negative Divided By Negative ...................... [PASS]
 [+]  29 Division Edge Round-Up (2/3) ...................... [PASS]
 [+]  30 High-Precision (No Cache) Pi Calculation .......... [PASS]
 [+]  31 High-Precision (Cached) Pi Fetch .................. [PASS]
 [+]  32 Zero Exponent for Power of 10 ..................... [PASS]
 [+]  33 Large Integer Exponent for Power of 10 ............ [PASS]
 [+]  34 Deep Negative Exponent for Power of 10 ............ [PASS]
 [+]  35 10 to Power of Negative Fractional ................ [PASS]
 [+]  36 10 to Power of Complex Fractional Exponent ........ [PASS]
 [+]  37 Explicit Positive Sign Exponent for Power of 10 ... [PASS]
 [+]  38 Dirty Integer Padded Exponent for Power of 10 ..... [PASS]
 [+]  39 10 to Power of Inverse Round-Trip Identity ........ [PASS]
 [+]  40 Clean Division With No Remainder .................. [PASS]
 [+]  41 Integer Division With Remainder ................... [PASS]
 [+]  42 Small Integer Modulo Large Integer ................ [PASS]
 [+]  43 Decimal Modulo With Half Remainder ................ [PASS]
 [+]  44 Small Decimal Modulo Large Decimal ................ [PASS]
 [+]  45 Remainder Check ................................... [PASS]
 [+]  46 High Precision Remainder .......................... [PASS]
 [+]  47 Positive Integer Modulo Negative Integer .......... [PASS]
 [+]  48 Negative Integer Modulo Positive Integer .......... [PASS]
 [+]  49 Negative Integer Modulo Negative Integer .......... [PASS]
 [+]  50 Is Zero Condition on True Zero .................... [PASS]
 [+]  51 Is Positive Condition on True Zero ................. [PASS]
 [+]  52 Is Negative Condition on True Zero ................. [PASS]
 [+]  53 Is Zero Condition on Tiny Positive ................. [PASS]
 [+]  54 Is Positive Condition on Tiny Positive ............. [PASS]
 [+]  55 Is Negative Condition on Tiny Positive ............. [PASS]
 [+]  56 Is Zero Condition on Negative ..................... [PASS]
 [+]  57 Is Positive Condition on Negative ................. [PASS]
 [+]  58 Is Negative Condition on Negative ................. [PASS]
 [+]  59 Is Equal Condition (Trailing Zeros Ignored) ....... [PASS]
 [+]  60 Is Not Equal Condition (Trailing Zeros Ignored) ... [PASS]
 [+]  61 Is Equal Condition on Mismatch .................... [PASS]
 [+]  62 Is Not Equal Condition on Mismatch ................ [PASS]
 [+]  63 Is Strictly Less Than Condition ................... [PASS]
 [+]  64 Is Strictly Greater Than Condition ................ [PASS]
 [+]  65 Is Less Than or Equal To Condiction ............... [PASS]
 [+]  66 Is Greater Than or Equal To Condition ............. [PASS]
 [+]  67 Is Less Than Condiction When Equal ................ [PASS]
 [+]  68 Is Greater Than Condition When Equal .............. [PASS]
 [+]  69 Is Less Than or Equal To Condiction When Equal .... [PASS]
```

```
[+]  70 Is Greater Than or Equal To Condition When Equal .. [PASS]
[+]  71 Positive Floor ..................................... [PASS]
[+]  72 Negative Floor ..................................... [PASS]
[+]  73 Floor with trailing zeros .......................... [PASS]
[+]  74 Floor of Zero ...................................... [PASS]
[+]  75 Floor Scale Larger Than Precision ................... [PASS]
[+]  76 Negative Floor Scale Larger Than Precision ......... [PASS]
[+]  77 Massive Positive Asymmetry Floor ................... [PASS]
[+]  78 Massive Negative Asymmetry Floor ................... [PASS]
[+]  79 Positive Ceiling ................................... [PASS]
[+]  80 Negative Ceiling ................................... [PASS]
[+]  81 Ceiling of Zero .................................... [PASS]
[+]  82 Positive Ceiling Scale -1 .......................... [PASS]
[+]  83 Negative Ceiling Scale -1 .......................... [PASS]
[+]  84 Massive Positive Asymmetry Ceiling ................. [PASS]
[+]  85 Massive Negative Asymmetry Ceiling ................. [PASS]
[+]  86 Standard Composite GCD ............................. [PASS]
[+]  87 GCD Of Coprime Integers ............................ [PASS]
[+]  88 Zero Edge GCD Case ................................. [PASS]
[+]  89 Negative Inputs GCD ................................ [PASS]
[+]  90 Scale Shift Check GCD .............................. [PASS]
[+]  91 Large Scale Stress GCD ............................. [PASS]
[+]  92 Fractional Match GCD ............................... [PASS]
[+]  93 Low Decimal Precision GCD .......................... [PASS]
[+]  94 GCD Of Trailing Zeros Ignored ...................... [PASS]
[+]  95 Negative Decimal Inputs GCD ........................ [PASS]
[+]  96 Standard Integers LCM .............................. [PASS]
[+]  97 LCM Of Coprime Integers ............................ [PASS]
[+]  98 Zero Edge Case LCM ................................. [PASS]
[+]  99 Fractional Decimals LCM ............................ [PASS]
[+] 100 Negative Inputs LCM ................................ [PASS]
[+] 101 Cast Integer Minimum Value ......................... [PASS]
[+] 102 Cast Long Maximum Value ............................ [PASS]
[+] 103 Cast Double Precision Decimal ...................... [PASS]
[+] 104 Cast String Clean Integer .......................... [PASS]
[+] 105 Cast String Padded Zeros ........................... [PASS]
[+] 106 Cast Malformed String Error Guard Rail ............. [PASS]
[+] 107 Cast Invalid Object Type Guard Rail ................ [PASS]
[+] 108 Rounding Up Without Leading Digits Before Dot ..... [PASS]
[+] 109 Rounding Up Test ................................... [PASS]
[+] 110 Rounding Down Test ................................. [PASS]
[+] 111 Embedded Internal Zeros ............................ [PASS]
[+] 112 Positive Truncate at Scale 1 ....................... [PASS]
[+] 113 Negative Truncate at Scale 1 ....................... [PASS]
[+] 114 Truncate Scale -1 Invariant ........................ [PASS]
[+] 115 Truncate Scale -2 Invariant ........................ [PASS]
[+] 116 Division by Zero Guard Rail ........................ [PASS]
[+] 117 Natural Logarithm of Zero Guard Rail ............... [PASS]
[+] 118 Natural Logarithm of Negative Guard Rail ........... [PASS]
[+] 119 Absolute Value of Negative Decimal ................. [PASS]
[+] 120 Absolute Value of Positive Decimal ................. [PASS]
[+] 121 Negate Positive Decimal ............................ [PASS]
[+] 122 Negate Negative Decimal ............................ [PASS]
[+] 123 Truncate Returns Integer Part ...................... [PASS]
[+] 124 Fraction Returns Fraction Part ..................... [PASS]
[+] 125 Truncate Negative Decimal .......................... [PASS]
[+] 126 Fraction Negative Decimal .......................... [PASS]
[+] 127 Parser Leading and Trailing Zeros .................. [PASS]
[+] 128 Parser Explicit Plus Sign .......................... [PASS]
[+] 129 Round to 4 Decimal Places .......................... [PASS]
[+] 130 Factorial of Small ................................. [PASS]
[+] 131 Factorial of Large Stress .......................... [PASS]
[+] 132 High-Precision Square Root of 2 .................... [PASS]
[+] 133 Power for Both Integer Base/Exponent ............... [PASS]
[+] 134 Power for Both Fractional Base/Exponent ............ [PASS]
[+] 135 Irrational Round-Trip Identity ..................... [PASS]
[+] 136 Fractional Round-Trip Identity ..................... [PASS]
[+] 137 Terminating Fractional Root ........................ [PASS]
[+] 138 Deep Precision Fractional Root ..................... [PASS]
[+] 139 Inverted Fractional Root ........................... [PASS]
[+] 140 Micro Irrational Fractional Root ................... [PASS]
```

```
 [+] 141 Basic Decimal Cubic ............................. [PASS]
 [+] 142 Precision Expansion Cubic ....................... [PASS]
 [+] 143 Negative Base Cubic ............................. [PASS]
 [+] 144 Massive Integer Carry Cubic ..................... [PASS]
 [+] 145 Zero Cubic Annihilation ......................... [PASS]
 [+] 146 Irrational Cubic Round-Trip Identity ............ [PASS]
 [+] 147 Negative Irrational Round-Trip Identity ......... [PASS]
 [+] 148 Perfect Cube Root ............................... [PASS]
 [+] 149 Terminating Fractional Cube Root ................ [PASS]
 [+] 150 Negative Input Cube Root ........................ [PASS]
 [+] 151 Precision Capped Cube Root ...................... [PASS]
 [+] 152 Micro Decimal Cube Root ......................... [PASS]
 [+] 153 Right Standard Shift ............................ [PASS]
 [+] 154 Right Shift From Zero Lead ...................... [PASS]
 [+] 155 Left Standard Shift ............................. [PASS]
 [+] 156 Left Shift To Zero Lead ......................... [PASS]
 [+] 157 Overshoot Padding Right Shift ................... [PASS]
 [+] 158 Integer Overshoot Right Shift ................... [PASS]
 [+] 159 Overshoot Padding Left Shift .................... [PASS]
 [+] 160 Tiny Decimal Overshoot Shift Left ............... [PASS]
 [+] 161 Shift Zero Count Identity ....................... [PASS]
 [+] 162 Shift Operating on Zero ......................... [PASS]
 [+] 163 Right Shift Negative Sign ....................... [PASS]
 [+] 164 Left Shift Negative Sign Overshoot .............. [PASS]
 [+] 165 Basic Positive Maximum .......................... [PASS]
 [+] 166 Argument Larger Maximum ......................... [PASS]
 [+] 167 Multiple Parameters Maximum ..................... [PASS]
 [+] 168 Mixed Signs Maximum ............................. [PASS]
 [+] 169 All Negative Maximum ............................ [PASS]
 [+] 170 High Precision Asymmetry Maximum ................ [PASS]
 [+] 171 Basic Positive Minimum .......................... [PASS]
 [+] 172 Base Object Smaller Minimum ..................... [PASS]
 [+] 173 Multiple Parameters Minimum ..................... [PASS]
 [+] 174 Mixed Signs Minimum ............................. [PASS]
 [+] 175 All Negative Minimum ............................ [PASS]
 [+] 176 High Precision Asymmetry Minimum ................ [PASS]
 [+] 177 Zero Boundary Check Minimum ..................... [PASS]

 ================================================================

    TESTS COMPLETE: 177 / 177 PASSED.
    *** ALL GREEN! No features broken.

Elapsed: 00:00:01.266


 ======= STARTING AUTOMATED TESTS ================================

 [+] 178 Logarithm Resolves to Exact Integer (No Cache) .... [PASS]
 [+] 179 High-Precision Logarithm (No Cache) ............... [PASS]
 [+] 180 High-Precision Natural Logarithm (No Cache) ....... [PASS]
 [+] 181 High-Precision Natural Logarithm of 10 (No Cache) . [PASS]
 [+] 182 Natural Logarithm of 1 Resolves to Zero ........... [PASS]
 [+] 183 Logarithm of 1 Resolves to Zero ................... [PASS]
 [+] 184 Natural Logarithm Round-Trip Identity ............. [PASS]
 [+] 185 Logarithm of Fractional Power of 10 ............... [PASS]
 [+] 186 Natural Logarithm of Fractional Value ............. [PASS]
 [+] 187 Logarithm of Very Large Integer ................... [PASS]
 [+] 188 Natural Logarithm of Large Arbitrary Integer ...... [PASS]
 [+] 189 Exponential of Zero ............................... [PASS]
 [+] 190 Exponential Round-Trip Identity ................... [PASS]
 [+] 191 High-Precision Exponential e ...................... [PASS]
 [+] 192 Sine of Zero Resolves to Zero ..................... [PASS]
 [+] 193 Sine Near Pi/2 Resolves to 1 ...................... [PASS]
 [+] 194 Sine Near Pi Resolves to Zero ..................... [PASS]
 [+] 195 Symmetry Test for Sine ............................ [PASS]
 [+] 196 High Precision Verification for Sine .............. [PASS]
 [+] 197 Cosine of Zero Resolves to 1 ...................... [PASS]
 [+] 198 Cosine Near Pi/2 Resolves to Zero ................. [PASS]
 [+] 199 Cosine Near Pi Resolves to -1 ..................... [PASS]
 [+] 200 Symmetry Test for Cosine .......................... [PASS]
```

```
 [+] 201 High Precision Verification for Cosine ..... [PASS]
 [+] 202 Tangent of Zero Resolves to Zero ..... [PASS]
 [+] 203 Tangent Near Pi Resolves to Zero ..... [PASS]
 [+] 204 Symmetry Test for Tangent ..... [PASS]
 [+] 205 High Precision Verification for Tangent ..... [PASS]
 [+] 206 Secant of Zero Resolves to 1 ..... [PASS]
 [+] 207 Secant Near Pi Resolves to -1 ..... [PASS]
 [+] 208 Symmetry Test for Secant ..... [PASS]
 [+] 209 High Precision Verification for Secant ..... [PASS]
 [+] 210 Cosecant Near Pi/2 Resolves to 1 ..... [PASS]
 [+] 211 Cosecant Near 3*Pi/2 Resolves to -1 ..... [PASS]
 [+] 212 Symmetry Test for Cosecant ..... [PASS]
 [+] 213 High Precision Verification for Cosecant ..... [PASS]
 [+] 214 Cotangent Near Pi/2 Resolves to Zero ..... [PASS]
 [+] 215 Symmetry Test for Cotangent ..... [PASS]
 [+] 216 High Precision Verification for Cotangent ..... [PASS]
 [+] 217 Sinh of Zero Resolves to Zero ..... [PASS]
 [+] 218 Symmetry Test for Sinh ..... [PASS]
 [+] 219 High Precision Verification for Sinh ..... [PASS]
 [+] 220 Cosh of Zero Resolves to 1 ..... [PASS]
 [+] 221 Symmetry Test for Cosh ..... [PASS]
 [+] 222 High Precision Verification for Cosh ..... [PASS]
 [+] 223 Tanh of Zero Resolves to Zero ..... [PASS]
 [+] 224 Symmetry Test for Tanh ..... [PASS]
 [+] 225 High Precision Verification for Tanh ..... [PASS]
 [+] 226 Fundamental Hyperbolic Identity ..... [PASS]
 [+] 227 180 Degrees to Pi Radians ..... [PASS]
 [+] 228 90 Degrees to Pi/2 Radians ..... [PASS]
 [+] 229 Zero Degrees to Zero Radians ..... [PASS]
 [+] 230 Negative Degrees to Negative Radians ..... [PASS]
 [+] 231 1 Radian to Degrees ..... [PASS]
 [+] 232 Pi Radians to 180 Degrees ..... [PASS]
 [+] 233 Zero Radians to Zero Degrees ..... [PASS]
 [+] 234 Negative Radians to Negative Degrees ..... [PASS]
 [+] 235 Logarithm Negative Guard Rail ..... [PASS]
 [+] 236 Tangent Pi/2 Asymptote Guard Rail ..... [PASS]
 [+] 237 Cosecant Zero Asymptote Guard Rail ..... [PASS]
 [+] 238 Secant Pi/2 Asymptote Guard Rail ..... [PASS]
 [+] 239 Cotangent Zero Asymptote Guard Rail ..... [PASS]

 ================================================================

    TESTS COMPLETE: 239 / 239 PASSED.
    *** ALL GREEN! No features broken.

Elapsed: 00:00:10.703
```